

2

ADDRESSING MODES

2.1 Introduction

The various addressing modes commonly used are:

Implied Mode:

In this mode the operands are specified implicitly in the definition of the instruction.

Example:

- Complement Accumulator (CMA) [All register reference instructions that use an accumulator are implied - mode instructions]
- Zero-address instructions in a stack organized computer. [here the operands are implied to be on top of the stack]

Immediate Mode:

In this mode the operand is specified in the instruction itself.

- It is very faster
- Useful for initializing register to a constant value.

Example:

ADD 10 i.e., $AC \leftarrow AC + 10$

- The range of value limited by the address field

Register Mode:

In this mode the operands are in registers that reside within the CPU.

- A K-bit field can specify any one of 2^k registers.
- Reduces the length of the address field.

Example:

ADD R1

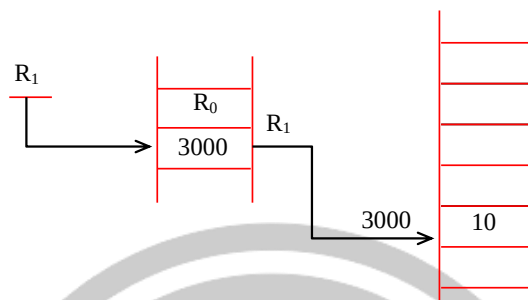
Register Indirect Mode:

In this mode the instruction specifies a register in the CPU whose contents give the address of an operand in memory.

- Before using this mode, the programmer must ensure that the memory address of the operand is placed in the processor register with a previous instruction.
- The advantage is the address field uses fewer bits to select or register.

Example:

ADD (R1)

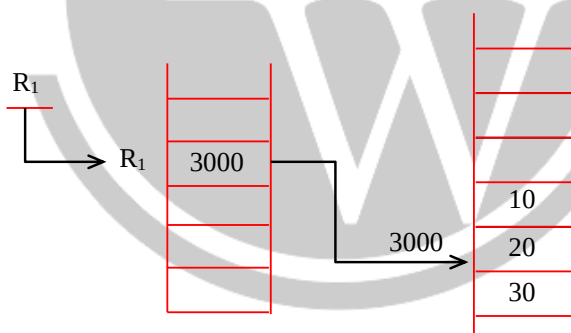


Autoincrement or Autodecrement Mode:

It is similar to register indirect mode except that the register is incremented or decremented after its value is used to access memory.

Example:

ADD (R1)

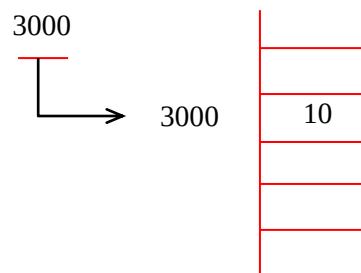


Direct Addressing Mode:

In this mode the effective address is equal to the address part of the instructions.

- The operand resides in memory and its address is given directly by the address field of the instruction.
- Used to represent global variables in a program.
- In a branch type instruction, the address field specifies the actual branch address space.
- Allows to access limited address space.
- The effective address is defined to be the memory address obtained from the computation dictated by the given addressing mode.
- The direct addressing mode is also called “Absolute mode”.

Example. ADD 3000



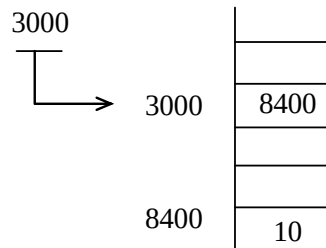
i.e. $AC \leftarrow AC + M[3000]$

Indirect Addressing mode:

In this mode, the address field of the instruction gives the address where the effective address is stored in memory.

- Allows to access larger address space
- Requires 2 memory cycles to read an operand

Example ADD 3000



2.1.1 Displacement addressing modes

The address field of the instruction added to the content of a specific register in the CPU. i.e,
Effective Address = Address part of instruction + content of Register.

2.1.2 The Various displacement addressing modes are

Relative addressing mode:

In this mode the content of program counter (PC) is added to the address part of the instruction.

- The address part is a signed number (2's complement) that can be either positive or negative.
- The result produces effective address whose position in memory is relative to the address of the next instruction.

$$EA = \text{Address Part (off set)} + PC \text{ value}$$

- Used with branch-type instructions when the branch address is in the area surrounding the instruction word itself.
- The advantage is, address field can be specified with a small number of bits compared to direct address.

Indexed Addressing mode:

In this mode the content of an index register is added to the address part of the instruction.

- Index register contains index value.
- Address part specifies the beginning address of a data array in memory.

$$EA = \text{Address Part (base address of data array)} + \text{Index register value (index value)}$$

- Used for accessing array of data.
- The index register can be special CPU register or any general purpose register.

Base register Addressing mode:

In this mode the content of a base register is added to the address part of the instruction.

- The base register is assumed to hold base address.
- The address field gives the displacement relative to this base address.

$$EA = \text{Address Part (displacement/offset)} + \text{Base register value (Base address)}$$

